数　据　结　构

林宏志

华中科技大学电信学院

linhz@hust.edu.cn

学时数：48（36＋12）

教　材：严蔚敏等，数据结构（C语言版），清华大学出版社，2021年6月 （配题集）

参考书：

[1] 高一凡，《数据结构》算法实现及解析（第二版），西电出版社，2004年10月

[2] 李春葆，新编数据结构习题与解析（第2版），清华大学出版社，2019年12月

[3] 殷人昆等，数据结构(用面向对象方法与C++语言描述第3版)，清华大学出版社，2021年9月

内 容 安 排

章	内 容	学时	章	内 容	学时
1	绪 论	2	7	图	5
2	线性表	5	8	动态存储管理	略
3	栈和队列	3	9	查找	3
4	串	3	10	内部排序	5
5	数组和广义表	3	11	外部排序	略
6	树和二叉树	7	12	文件	略

第1-9周，周二3-4节 东九楼B503

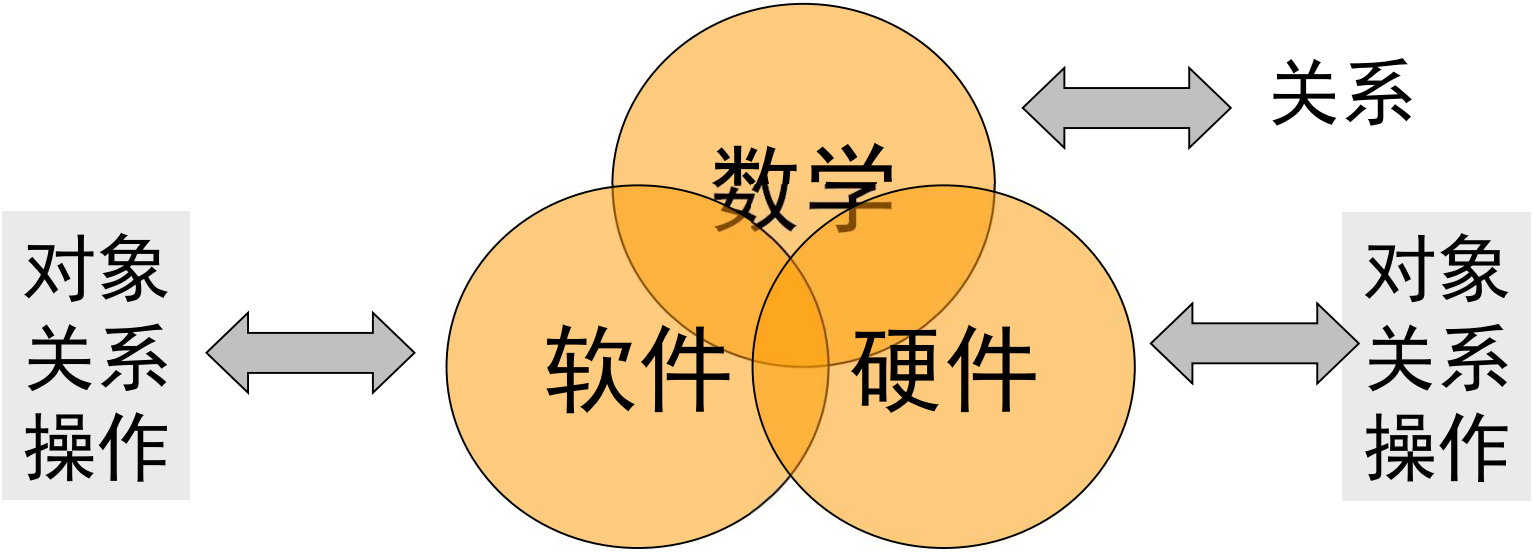
第1-9周，周五5-6节 东九楼B503

数据结构课程的重要性

——针对非数值计算的程序设计问题，研究计算机的操作对象以及它们之间的关系和操作。

——是介于数学、计算机硬件和计算机软件三者之间的一门核心课程。

Data_Structure=（D, R）



目　　录

第1章　绪论
第2章　线性表
第3章　栈和队列
第4章　串
第5章　数组和广义表
第6章　树和二叉树
第7章　图
第9章　查找
第10章　排序

第1章　绪　论

讨论5个问题：

1.1 什么是数据结构

1.2 学习数据结构的意义

1.3 数据结构涵盖的主要内容

1.4 什么是抽象数据类型

1.5 算法效率的度量

1.1 什么是数据结构

是相互之间存在一种或多种特定关系的数据元素的集合，表示为：

Data_Structure=（D, R）

（*数值或非数值*）

元素有限集

关系有限集

——*是指同一数据元素类型中各元素之间存在的关系*

术语简介：数据、数据元素和数据项

数据(data)——所有能被计算机识别、存储和处理的符号的集合（包括数字、字符、声音、图像等信息 ）。

数据元素(data element)——是数据的基本单位，具有完整确定的实际意义(又称元素、结点，顶点、记录等）。

数据项(Data item)——构成数据元素的项目。是具有独立含义的最小标识单位（又称字段、域、属性 等）。

三者之间的关系：数据 > 数据元素 > 数据项

例：班级通讯录 > 个人记录 > 姓名、年龄……

1.2 学习数据结构的意义

计算机内的数值运算依靠方程式，而非数值运算（如表、树、图等）则要依靠数据结构。

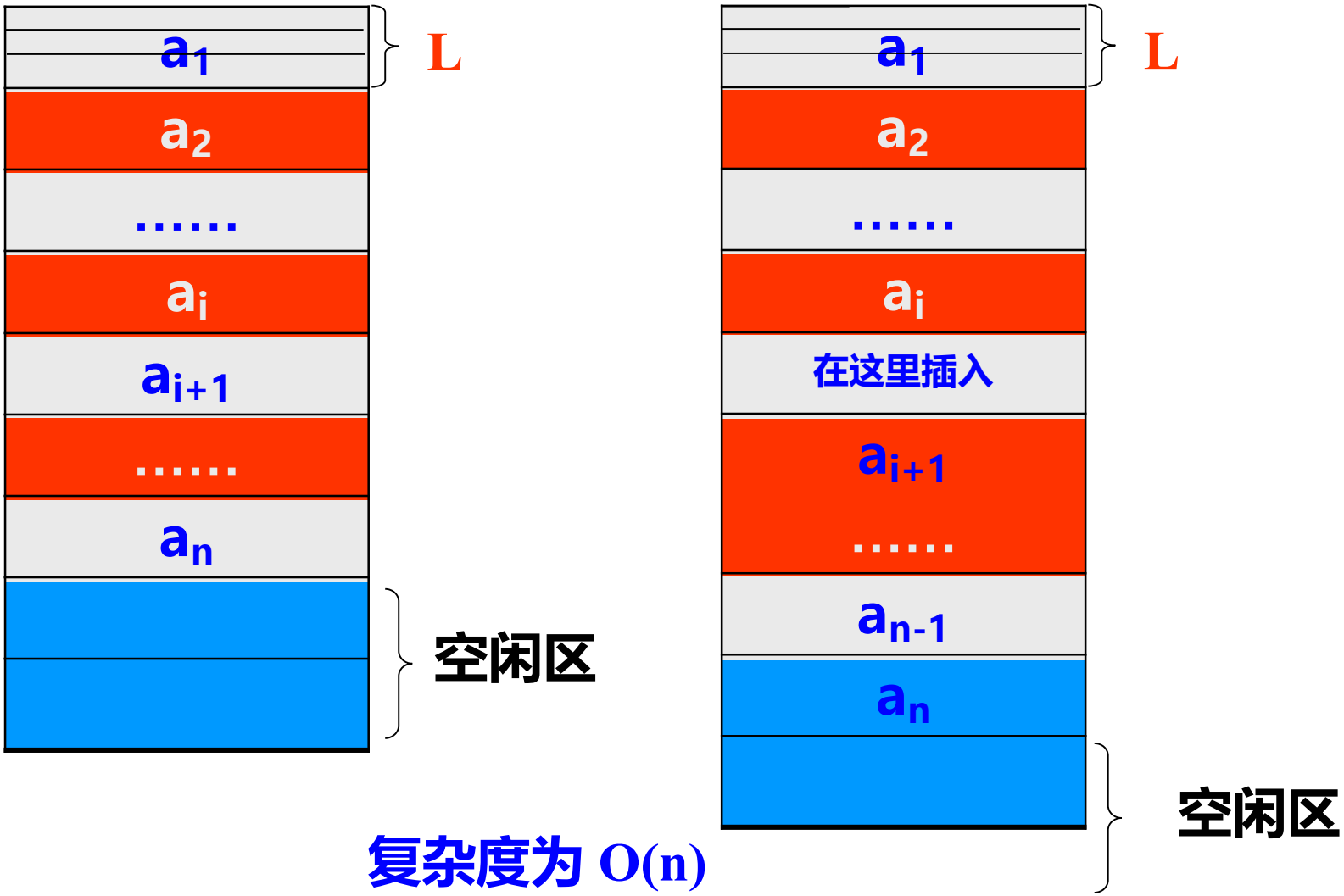
数据结构是一门学科，针对非数值计算的程序设计问题，研究计算机的操作对象以及它们之间的关系和操作等等。

同样的数据对象，用不同的数据结构来表示，运算效率可能有明显的差异。

程序设计＝好算法＋好结构

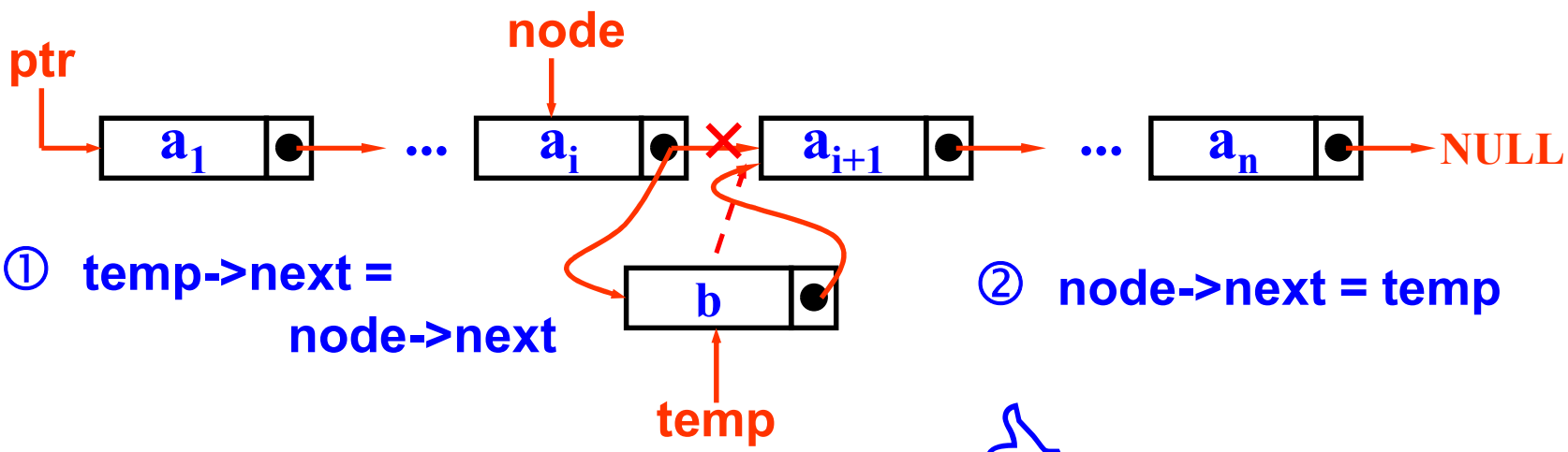
下面我们举四个例子来说明

1. 线性表的顺序存储结构插入示意图



复杂度为 O(n)

线性表的链式存储结构插入示意图



① temp->next = node->next

② node->next = temp

复杂度为 O(1)

2. 乘法可以计算的更快吗?

- 8902 ✖ 3921
- =
- 8902
- 17804
- 80118
- 26706
- = 34904742

O(n^2)

分解为更小的数的乘法

- X * Y
- 8902 * 3921
- a b * c d

- X*Y = (100a+b)*(100c+d)
- =10000a*c+100(a*d+b*c)+ b*d
- 这样会更快吗？

答案是否定的

- 因为每次要计算四个小的乘法
- 每个小乘法的复杂度是 (n/2)^2
- 总的计算量为 4*(n^2)/4= n^2
- 跟没有分解之前的计算量相同
- 那是否意味着乘法运算没有办法计算的更快？
- 也就是说，小乘法的个数能否小于4？

小乘法的个数能否小于4?

- X*Y = (100a+b)*(100c+d)
- =10000a*c+100(a*d+b*c)+ b*d
-
- a*d+b*c = (a+b)*(c+d)- a*c - b*d
- 这样只需要做3个小乘法
- 最终的复杂度为O(n^1.58)

3. 跟大家玩一个点名的游戏

- 想知道有多少人来上课
- 每个人依次报数，最后一个人报的数为实到人数。
- 有没有更快的算法？

- 所有人起立，并给每个人赋予一个初始值1；
- 任意找一个同伴，并将自己的数字和同伴的相加，得到的结果由其中一个人记住作为自己的数，而另一个人坐下；
- 每次都是由站着的同学重复以上操作；
- 直到最后一个站着的人报告他的数字，就是今天来上课的人数。
- 大家估计一下需要多少次能统计出人数，跟依次报数比较效率能有多少提升？

4. 同一个问题，比较四种不同算法效率差异

- 给定一组数字（有可能是负数） $A_1, A_2, \ldots, A_N$, 求最大的连续和，即max $\sum_{k=i}^{j} A_k$.
- 以上就是i循环1~N，j循环1~N，k循环从i~j，然后算出的结果保留最大值就行。

算法一

```
int  MaxSubsequenceSum ( const int A[ ],  int  N )
{
            int  ThisSum,  MaxSum,  i,  j,  k;
/* 1*/      MaxSum = 0;   /* initialize the maximum sum */
/* 2*/      for( i = 0; i < N; i++ )  /* start from A[ i ] */
/* 3*/          for( j = i; j < N; j++ ) {   /* end at A[ j ] */
/* 4*/                  ThisSum = 0;
/* 5*/                  for( k = i; k <= j; k++ )
/* 6*/                      ThisSum += A[ k ];  /* sum from A[ i ] to A[ j ] */
/* 7*/                  if ( ThisSum > MaxSum )
/* 8*/                      MaxSum = ThisSum;  /* update max sum */
                    }  /* end for-j and for-i */
/* 9*/      return  MaxSum;
}
```

$$T(N) = O(N^3)$$

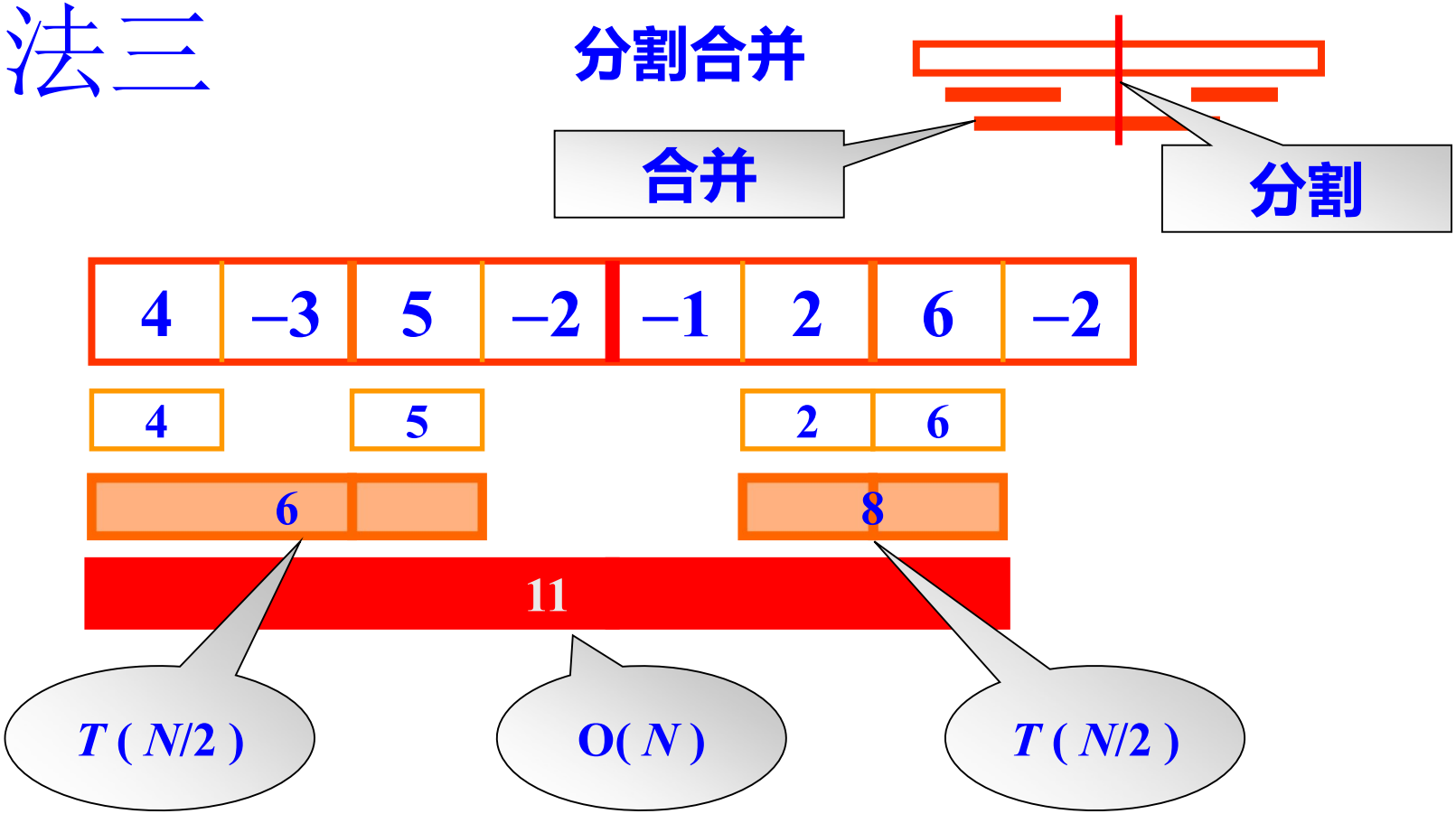
算法二

```
int  MaxSubsequenceSum ( const int A[ ],  int  N )
{
            int  ThisSum,  MaxSum,  i,  j;
/* 1*/      MaxSum = 0;   /* initialize the maximum sum */
/* 2*/      for( i = 0; i < N; i++ )  {   /* start from A[ i ] */
/* 3*/          ThisSum = 0;
/* 4*/          for( j = i; j < N; j++ ) {   /* end at A[ j ] */
/* 5*/              ThisSum += A[ j ];  /* sum from A[ i ] to A[ j ] */
/* 6*/              if ( ThisSum > MaxSum )
/* 7*/                  MaxSum = ThisSum;  /* update max sum */
            }  /* end for-j */
            }  /* end for-i */
/* 8*/      return  MaxSum;
}
```

$$\sum_{k=i}^{j} A_k = A_{jk} + \sum_{k=i}^{j-1} A_k$$

$$T(N) = O(N^2)$$

算法三



$$T(N) = 2\,T(N/2) + c\,N, \qquad T(1) = O(1)$$

$$= 2\,[2\,T(N/2^2) + c\,N/2] + c\,N$$

$$= 2^k\,O(1) + c\,k\,N \qquad \text{where } N/2^k = 1$$

$$= N\,O(1) + cN \log N$$

$$= O(N \log N)$$

算法四

```
int MaxSubsequenceSum( const int  A[ ],  int  N )
{
            int  ThisSum, MaxSum, j;
/* 1*/      ThisSum = MaxSum = 0;
/* 2*/      for ( j = 0; j < N; j++ ) {
/* 3*/          ThisSum += A[ j ];
/* 4*/          if  ( ThisSum > MaxSum )
/* 5*/              MaxSum = ThisSum;
/* 6*/          else if ( ThisSum < 0 )
/* 7*/              ThisSum = 0;
            }  /* end for-j */
/* 8*/      return MaxSum;
}
```

–1	3	–2	4	–6	1	6	–1

可见不同算法的性能差距非常大

$$T(N) = O(N)$$

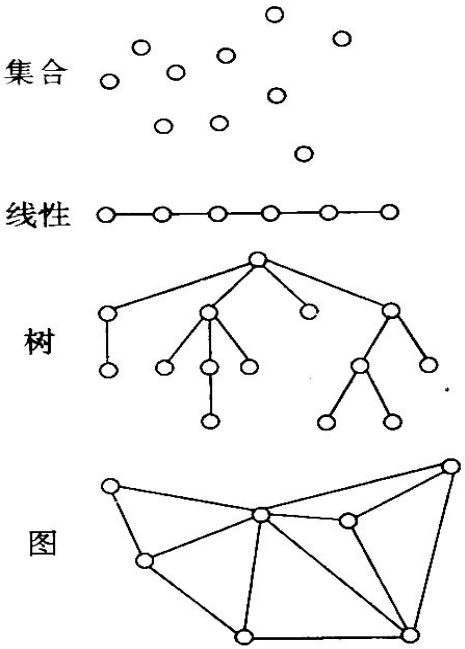
1.3 数据结构涵盖的内容

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈 、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

解释1： 什么叫数据的逻辑结构？

答：指数据元素之间的逻辑关系。即从逻辑关系上描述数据，它与数据的存储无关，是独立于计算机的。

逻辑结构可细分为4类：



集合结构： 仅同属一个集合

线性结构： 一对一（1:1） → 线 性

树 结 构： 一对多（1:n）

图 结 构： 多对多 (m:n)

} 非线性

例：用图形表示下列数据结构，并指出它们是属于线性结构还是非线性结构。

（1） S=(D, R)

D={ a, b, c, d, e, f }

R={(a,e), (b,c), (c,a), (e,f), (f,d)}

解： 上述表达式可用图形表示为：

$$b \longrightarrow c \longrightarrow a \longrightarrow e \longrightarrow f \longrightarrow d$$

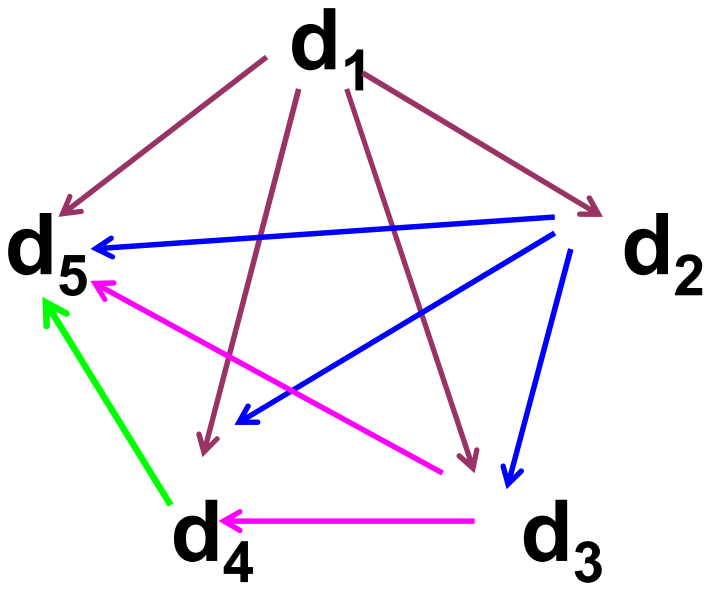
此结构为线性的。

（2） S=(D, R)

$D=\{d_i \mid 1 \le i \le 5\}$

$R=\{(d_i, d_j), i<j\}$

解：上述表达式可用图形表示为：



该结构是非线性的。

解释2：什么叫数据的物理结构？

答：物理结构亦称存储结构，是数据的逻辑结构在计算机存储器内的表示（或映像）。它依赖于计算机。

存储结构可分为4大类：　**顺序、链式、索引、散列**

例：复数**3.0－2.3i** 的两种存储方式：

法1：

地址	内容
0300	3.0
0302	－2.3

} 2字节

法2：

地址	内容
0300	3.0
0302	0415
0415	**－2.3**

解释3：什么是数据的运算？

答：在数据的逻辑结构上定义的操作算法。

它在数据的存储结构上实现。

最常用的数据运算有 5 种：

插入、删除、修改、查找、排序

1.4 什么是抽象数据类型

抽象数据类型和伪码是学习数据结构的工具

讨论：

1.4.1 数据类型与抽象数据类型的区别？

1.4.2 抽象数据类型如何定义？

1.4.3 抽象数据类型如何表示和实现？

1.4.1 数据类型与抽象数据类型的区别

数据类型： 是一个值的集合和定义在该值上的一组操作的总称。

抽象数据类型： 由用户定义的一个数学模型与定义在该模型上的一组操作，它由基本的数据类型构成。

ADT =Abstract Data Type

ADT与数据类型实质上是一个概念，但其特征是使用与实现分离，实行封装和信息隐蔽（独立于计算机）

“抽象”的意义在于数据类型的数学抽象特性

1.4.2 抽象数据类型如何定义

抽象数据类型可以用以下的三元组来表示：

ADT = （D，R，P）

数据对象　　D上的关系集　D上的操作集

ADT 常用定义格式

ADT抽象数据类型名{

数据对象：<数据对象的定义>

数据关系：<数据关系的定义>

基本操作：<基本操作的定义>

} ADT抽象数据类型名

例1：给出自然数(***Natural Number***)的抽象数据类型定义。

“抽象”的意义在于数据类型的数学抽象特性

ADT *Natural_Number* {

objects: 一个整数的有序子集合，它开始于0，结束于机器能表示的最大整数（*MAX INT*）

functions: 对于所有的 x, y ∈ Natural_Number； TRUE, FALSE ∈ Boolean； +, −, <, = = ,=等都是可用的服务。

Zero ( ): Natural Number 返回 0

IsZero(x): Boolean if (x==0) 返回TRUE else 返回 FALSE

Add(x, y): Natural Number if (x+y <= MAX INT)返回 x+y
else 返回 MAX INT

Subtract(x,y): Natural Number if (x<y)返回0 else 返回x-y

Equal(x,y): Boolean if (x== y)返回TRUE else 返回FALSE

Successor(x): Natural Number if (x == *MAX INT*)返回x else 返回x+1

} *Natural_Number*

1.4.3 抽象数据类型如何表示和实现

抽象数据类型可以通过固有的数据类型（如整型、实型、字符型等）来表示和实现。

注1：它有些类似C语言中的结构（struct)类型，但增加了相关的服务(或操作)。

注2：教材中用类C语言（介于伪码和C语言之间）作为描述工具。其描述语法汇总在教材P10-11上。

但上机时要用具体语言实现，如C或C++等

例2：给出抽象数据类型三元组（**Triplet**）的定义。

提示：

教材中例1-6和例1-7分别给出了抽象数据类型“三元组”的定义、表示和实现，请自己先试读一遍。

当课程学到50%以后，你再回头看这个例子，会发现自己已能完全看懂了！

例2：给出抽象数据类型三元组（Triplet）的定义。

“抽象”的意义在于数据类型的数学抽象特性

ADT Triplet {

数据对象： D={e1,e2,e3 | e1,e2,e3 ∈ ElemSet, （定义了关系运算的某个集合）}

数据关系： R1={<e1, e2 >, <e2, e3 >}

基本操作：

InitTriplet(&T,v1,v2,v3) //建三元组，给e1,e2,e3赋初值

DestroyTriplet(&T) //销毁三元组

Get(T, i, &e) //读取第i元的值输出给e

Put(&T,i,e) //修改第i元的值=e

IsAscending(T) //三元素单调增则返回1,否则返回0

IsDscending(T) //三元素单调减，则返回1,否则返回0

Max(T, &e) //求三元素中的最大值并输出给e

Min(T, &e) //求三元素中的最小值并输出给e

} ADT Triplet

1.5 算法效率的度量

讨论：

1.5.1 什么是算法？如何评判算法的好坏？

1.5.2 时间复杂度和空间复杂度如何表示？

1.5.3 计算举例

1.5.1 什么是算法？如何评判一个算法的好坏？

算法是对特定问题求解步骤的一种描述，它是指令的有限序列，是一系列输入转换为输出的计算步骤。

程序设计的实质：好算法＋好结构

算法的基本特性：有穷性、确定性、可行性、必有输出

算法评价指标：

正确性、可读性、健壮性、效率与低存储量需求

常用时间复杂度来衡量　　常用空间复杂度来衡量

1.5.2 时间复杂度和空间复杂度如何表示？

多项式阶

时间复杂度T(n)按数量级递增顺序为：

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	…	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

复杂度低 → 复杂度高

注：

1） O（）为渐近符号。

2） 空间复杂度S(n)按数量级递增顺序也与上表类似。

*渐进符号（O）的定义：*当且仅当存在一个正的常数 C，使得对所有的 $n \geq n_0$，有 $f(n) \leq Cg(n)$，则：

$f(n) = O(g(n))$

例：

$3n+2=O(n)$ 因为 $3n+2 \leq 4n$ for $n \geq 2$

$6*2^n+n^2=O(2^n)$ 因为 $6*2^n+n^2 \leq 7*2^n$ for $n \geq 4$

1.5.3 计算举例

例：分析以下程序段的时间复杂度。

```
i=1;            ①
while(i<=n)
    i=i*2;      ②
```

解：该算法的运行时间由程序中所有语句的频度（即该语句重复执行的次数）之和构成。

算法的时间复杂度由嵌套最深层语句的频度决定

分析：显然，语句①的频度是1。设语句2的频度是f(n)，则有：

$2^{f(n)} \le n$ 即$f(n) \le \log_2 n$，取最大值$f(n)=\log_2 n$

所以该程序段的时间复杂度$T(n)=1+f(n)=1+\log_2 n=O(\log_2 n)$

本章小结

数据结构课程—— 数据结构 + 算法 = 程序，涉及数学、计算机硬件和软件。

数据结构定义——指互相有关联的数据元素的集合，可用data_Structure=(D,R)表示。

数据结构内容——数据的逻辑结构、存储结构和基本运算(计算机处理非数值对象)

数据结构学习工具——抽象数据类型和伪码（类C）

算法效率指标——时间效率和空间效率

第1章结束

作业：

① 配套习题集的**1.6 1.7 1.8 1.10 1.17 1.20** 题。

② 复习**C**语言，重点是结构类型和递归概念。

学习方法：

① **1**上课课时**+**至少**2**课时课后复习

② 课程要求高，特别要先把**C**语言要学好，需要自己补课

③ 在力所能及情况下，把所有习题都自己做一遍

④ 缺课？考前突击？

补充4个问题

- 关注问题本身和解决方案的思路，而不是解决方案的细节；
- 我们的课程不同于编程课，更多的是理念和方法；
- 编程只是一种工具，所以我们不深究编程语言的细节，而是正确使用即可。

（1）需要多少人？

- 在沙漠里有100瓶水，其中一瓶是辣椒水，这瓶水会让人尝了觉得有辣味。
- 请问：如果一个人只能品尝一次（因为喝了就辣到失去味觉了，不能继续尝了），需要多少人品尝才能找到哪一瓶是辣椒水？

思路

- 不难发现，如果100人每人品尝相应的瓶子的水，问题就迎刃而解了，但这并不是一个好答案！

解决方案

- 我们知道7bit数字可以表示0-127，所以我们用7个人，理论上可以区别100瓶水，但是问题是该怎么做？

解决方案（续）

- 我们给7个人，每个人起名字为：a，b，c，d，e，f，g。
- 给出1-100瓶二进制表达式，如10是0001010
- 如果对应的bit位置是1，则将瓶子混合到对应人的杯子中

解决方案（续）

- 例如10号瓶子可以表示为0001010，所以这个瓶子的水，就滴入到d和f的杯子中
- 最后根据7个人报告他们所感觉的味道，例如，如果a，c，e觉得辣，1010100是84，我们就知道第84瓶中的水有毒

举例

- a b c d e f g
- 1=0 0 0 0 0 0 1
- 2=0 0 0 0 0 1 0
- 3=0 0 0 0 0 1 1
- 4=0 0 0 0 1 0 0
- 5=0 0 0 0 1 0 1
- 6=0 0 0 0 1 1 0
- 7=0 0 0 0 1 1 1

- 将第一瓶水滴到g的杯子中
- 2号瓶子的水滴到 f的杯子中
- 3号瓶子的水滴到f，g杯子中
- 4号瓶子到 e
- 5号瓶子到 e g
- 6号瓶子到 e f
- 7号瓶子到 e f g

思考

- 观察二进制表达式，如果那个地方的数字是1，则对应的人的杯子中就有该瓶的水
- 如果该位置的数字为0，则不影响结果
- 本质上，就是“与”运算

另一种解法

- 将前50瓶各倒一点混合在一起，并派一个人品尝，如果有辣味，则在前50瓶中，再选择前25瓶混合，如果没有辣味，那么辣椒水在后50瓶中，从后50瓶选择前25混合
- 继续以上操作，直到找到哪一瓶是辣椒水

（2）寻找最大和次大值

- 很显然，寻找最大的数很容易，我们可以将数字两两比较，保留大的那个，那么当比较完成，则找到最大的一个；
- 那么一个容易想到的方法是，可以把除最大值以外的数再用同样的方法比较一次，则可以找出次大值

代码实现

- #include <stdio.h>
  void main() {
  - int max=a[0]; numofa 是 a 中的元素个数
- for(i=1;i<numofa;i++){
  - if(max<a[i]){
  - max=a[i];
  - place=i;
  - }
  - }
- } // 需要 numofa 次

找次大值

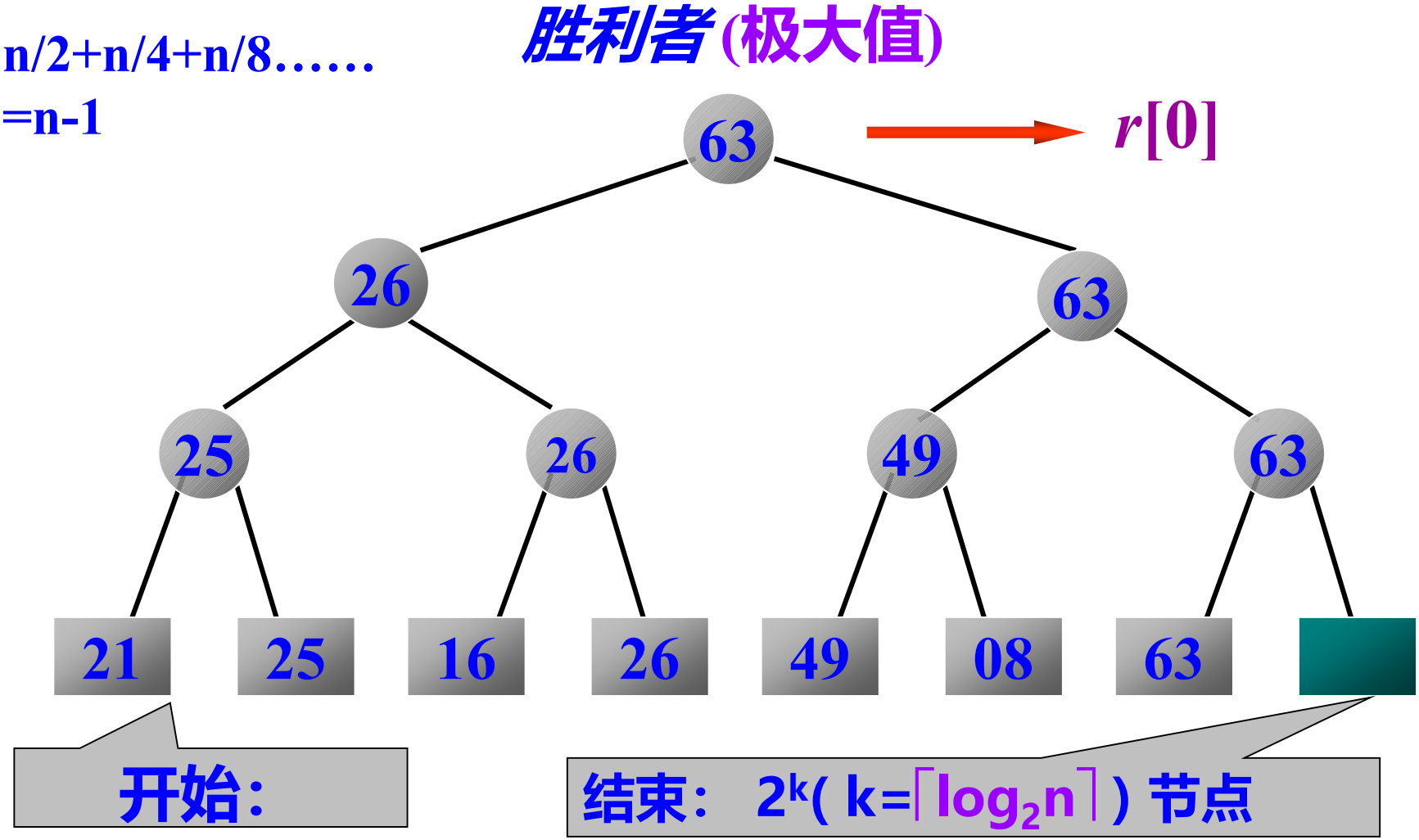
- temp= a[0]; a[0]=a[place];a[place]=temp;
- // 交换 max 和 a[0]
- Max2=a[1];
- for(i=2;i<numofa;i++){
  - if(max2<a[i])
  - max2=a[i];
- } // 需要 numofa-1 次

有没有更好的方法？

- 以上方法并不是最佳的方式，因为第一次找最大值的时候，我们获得了一些信息，所以找次大值的时候我们可以进行优化

更好的方法

n/2+n/4+n/8……
=n-1



怎么找次大值

- 次大值并不在比较树的高层，所以我们不能通过搜索高层来找到次大值
- 但是我们仔细观察这个比较树会发现，次大值一定跟最大值比较过
- 因此我们只需要 $k=\lceil \log_2 n \rceil$ 次就可以找到次大值， **k**其实就是这棵树的高度

复杂度比较

- 第一种方法: n-1+n-2
- 改进算法: n-1+ $\log_2 n$

（3）需要运行多少次才能停下来?

- X 是一个正的整数，下面的函数需要运行多少次才能停下来？

$$f(x)=\begin{cases} x/2 & \text{如果 } x \text{ 是偶数} \\ f(\,f(3x+1)\,) & \text{如果 } x \text{ 是奇数} \end{cases}$$

X=1　f(x)=f(f(3+1))=f(2)=1　　3 次

X=2　f(x)=f(2)=1　　1 次

X=3　f(x)=f(f(10))=f(5)=ff(16)=f(8)=4

X=4　f(x)=2

X=5　。。。。。。

X=17　f(x) =ff(52)=f(26)=13

X=31　f(x)=ff(94)=f(47)=ff(142)=f(71)

=ff(214)=f(107)=ff(322)=f(161)

=ff(484)=f(242)=121

我们得到什么启示？

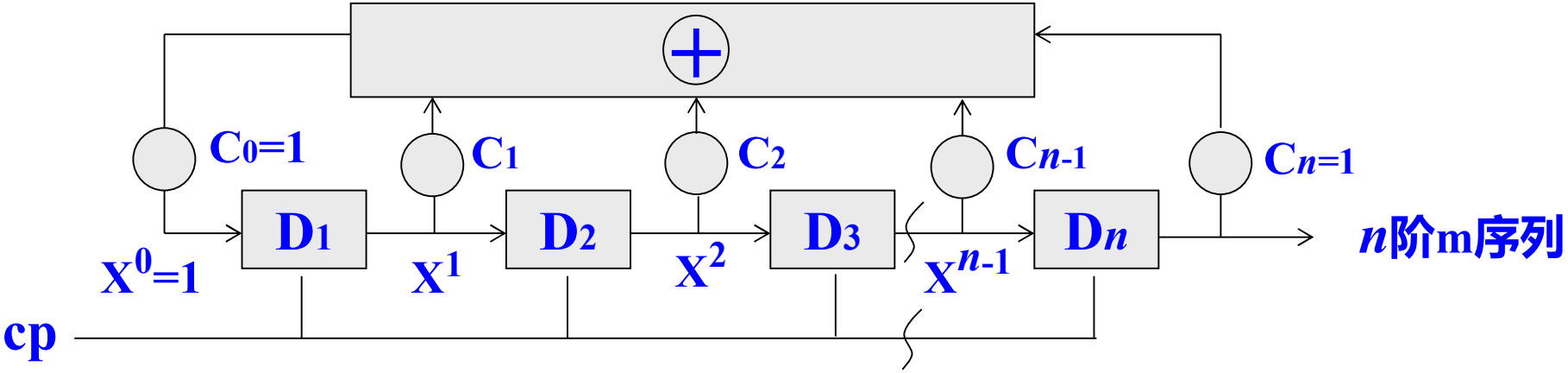
思考

- X= 4n or x= 4n+2　1 次
- X= 4n+1 $f(f(3x+1))$ = $f(f(12n+4))$ = $f(6n+2)$ = 3n+1　3 次
- X= 4n+3 $f(f(3x+1))$ = $f(f(12n+10))$ = $f(6n+5)$ 需要进一步运行
- 设n=4m+1 or n=4m+3 继续以上运算
- 所以运行次数p依赖如下关系 $x=2^p-1$

证明

- $X=2^p-1$
- $f(\,f(3x+1)\,)=f(\,f(3*2^p-2)\,)$
- $=\ f\left(3*2^{p-1}-1\right)$
- $=\ f(\,f\left(3^2*2^{p-1}-2\right)\,)$
- $=\ f\left(3^2*2^{p-2}-1\right)$
- $=\ \ldots\ldots\ldots$
- $=\ f(3^p-1)=(3^p-1)/2$

（4） m序列发生器的一般构成



(1). 构成：将移位寄存器某些级(反馈系数C*i*=1的级)模2加后反馈至输入级，在时钟驱动下产生m序列。

(2). 生成多项式 $f(x)=1+C_1X+C_2X^2+\ldots C_{n-1}X^{n-1}+X^n$

除了采用递推一个个的计算，
对任意指定的位置的值，有什么快速算法?